

✓ 1. Simulador de Neurônio Artificial ou Perceptron

✓ 1.1. Primeiro código com os pesos dados e apenas 2 casos para serem classificados

- **Pesos e Bias Predefinidos:** Aqui, o peso associado ao "peso da fruta" é 0.02 e para "cor" é 1.5, com um bias de -1.5. Esses valores foram escolhidos para diferenciar as frutas sem treinamento.
- **Disparo do neurônio:** A função `step_function` é tal que se o valor `x` for maior do que 0, então, o neurônio "dispara" e irá retornar o valor 1, senão o valor 0 será retornado.
- **Classificação:** A função `classify` calcula a soma ponderada e aplica a função de ativação `step_function`, que classifica a fruta como "Laranja" ou "Maçã" com base na saída binária (0 ou 1).

```
import numpy as np

# Dados de pesos e bias predefinidos para separar laranjas de maçãs
# Vamos assumir que o peso para "peso da fruta" é 0.02 e para "cor" é 1.5, e que o bias é -1.5
weights = np.array([0.02, 1.5])
bias = -1.5

# Função de ativação (degrau)
def step_function(x):
    return 1 if x >= 0 else 0

# Função para prever a classificação com pesos fixos
def classify(fruit_weight, fruit_color):
    inputs = np.array([fruit_weight, fruit_color])
    summation = np.dot(inputs, weights) + bias
    result = step_function(summation)
    print('result', result)
    return "Laranja" if result == 1 else "Maçã"

# Exemplos de teste
print("Peso 155g e cor 0.6 é uma:", classify(155, 0.6)) # Espera-se "Maçã"
print("Peso 170g e cor 0.8 é uma:", classify(170, 0.8)) # Espera-se "Laranja"
```

```
↗ result 1
Peso 155g e cor 0.6 é uma: Laranja
result 1
Peso 170g e cor 0.8 é uma: Laranja
```

✓ 1.2. Segundo código: Visualização das 5 amostras de laranjas e 5 de maçãs, bem como do hiperplano separador atrelado aos pesos dados

```
import numpy as np
import matplotlib.pyplot as plt

# Pesos e bias predefinidos
weights = np.array([0.02, 1.5])
bias = -1.5

# Função de ativação (degrau)
def step_function(x):
    return 1 if x >= 0 else 0

# Função para prever a classificação
def classify(fruit_weight, fruit_color):
    inputs = np.array([fruit_weight, fruit_color])
    summation = np.dot(inputs, weights) + bias
    result = step_function(summation)
    return "Laranja" if result == 1 else "Maçã"

# Dados de exemplo
# Pontos de maçãs (peso, cor)
macas = np.array([
    [130, 0.5],
    [140, 0.55],
    [120, 0.45],
    [135, 0.52],
    [125, 0.48]
])
```

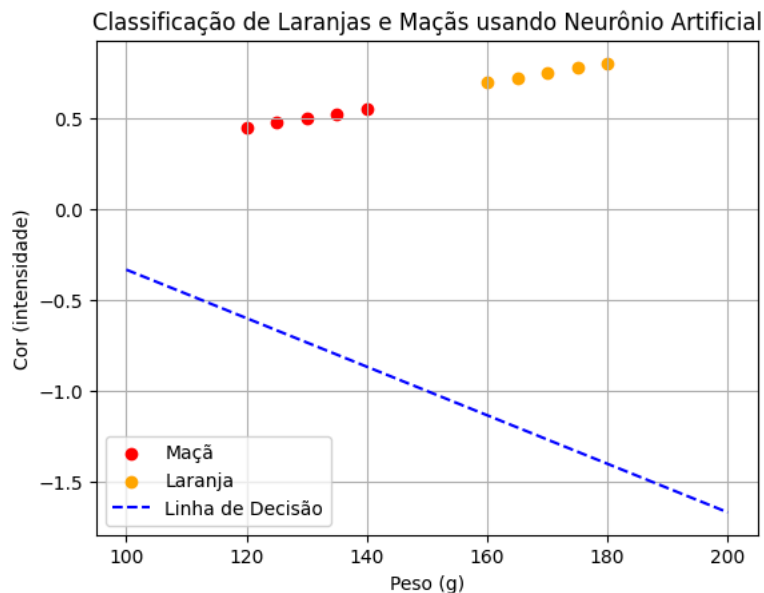
```
# Pontos de laranjas (peso, cor)
laranjas = np.array([
    [160, 0.7],
    [170, 0.75],
    [180, 0.8],
    [165, 0.72],
    [175, 0.78]
])

# Plotando os pontos
plt.scatter(macas[:, 0], macas[:, 1], color="red", label="Maçã")
plt.scatter(laranjas[:, 0], laranjas[:, 1], color="orange", label="Laranja")

# Linha de decisão: resolvemos a equação  $\theta = \text{weight1} \cdot x + \text{weight2} \cdot y + \text{bias}$  para y em função de x
x_values = np.linspace(100, 200, 100)
y_values = -(weights[0] * x_values + bias) / weights[1]
plt.plot(x_values, y_values, color="blue", linestyle="--", label="Linha de Decisão")

# Configurações do gráfico
plt.xlabel("Peso (g)")
plt.ylabel("Cor (intensidade)")
plt.title("Classificação de Laranjas e Maçãs usando Neurônio Artificial")
plt.legend()
plt.grid(True)
plt.show()

# Teste de classificação
print("Peso 155g e cor 0.6 é uma:", classify(155, 0.6))
print("Peso 170g e cor 0.8 é uma:", classify(170, 0.8))
```



✓ 1.3. Código com Bias e pesos ajustados para separar corretamente os dois conjuntos de dados

```
import numpy as np
import matplotlib.pyplot as plt

# Pesos e bias ajustados para separar corretamente laranjas e maçãs
weights = np.array([0.03, 1.0]) # Ajuste nos pesos
bias = -5.2 # Ajuste no bias

# Função de ativação (degrau)
def step_function(x):
    return 1 if x >= 0 else 0

# Função para prever a classificação
def classify(fruit_weight, fruit_color):
    inputs = np.array([fruit_weight, fruit_color])
    summation = np.dot(inputs, weights) + bias
    result = step_function(summation)
    #print('result', result)
    return "Laranja" if result == 1 else "Maçã"

# Dados de exemplo
```

```
# Pontos de maçãs (peso, cor)
macas = np.array([
    [130, 0.5],
    [140, 0.55],
    [120, 0.45],
    [135, 0.52],
    [125, 0.48]
])

# Pontos de laranjas (peso, cor)
laranjas = np.array([
    [160, 0.7],
    [170, 0.75],
    [180, 0.8],
    [165, 0.72],
    [175, 0.78]
])

# Plotando os pontos
plt.scatter(macas[:, 0], macas[:, 1], color="red", label="Maçã")
plt.scatter(laranjas[:, 0], laranjas[:, 1], color="orange", label="Laranja")

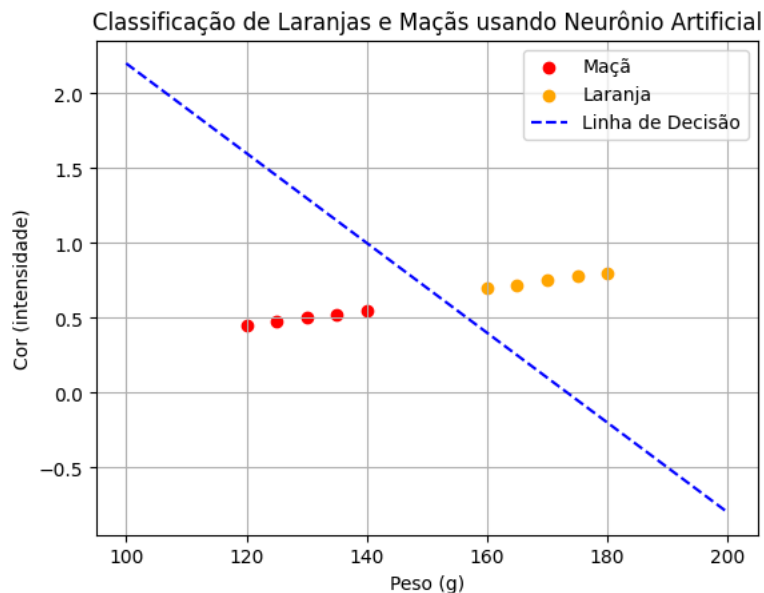
# Linha de decisão: resolvemos a equação  $\theta = \text{weight1} \cdot x + \text{weight2} \cdot y + \text{bias}$  para y em função de x
x_values = np.linspace(100, 200, 100)
y_values = -(weights[0] * x_values + bias) / weights[1]
plt.plot(x_values, y_values, color="blue", linestyle="--", label="Linha de Decisão")

# Configurações do gráfico
plt.xlabel("Peso (g)")
plt.ylabel("Cor (intensidade)")
plt.title("Classificação de Laranjas e Maçãs usando Neurônio Artificial")
plt.legend()
plt.grid(True)
plt.show()

# Teste de classificação

# Para as laranjas
for laranja in laranjas:
    print(f"Peso {laranja[0]}g e cor {laranja[1]} é uma:", classify(laranja[0], laranja[1]))

# Para as maçãs
for maca in macas:
    print(f"Peso {maca[0]}g e cor {maca[1]} é uma:", classify(maca[0], maca[1]))
```



```
Peso 160.0g e cor 0.7 é uma: Laranja
Peso 170.0g e cor 0.75 é uma: Laranja
Peso 180.0g e cor 0.8 é uma: Laranja
Peso 165.0g e cor 0.72 é uma: Laranja
Peso 175.0g e cor 0.78 é uma: Laranja
Peso 130.0g e cor 0.5 é uma: Maçã
Peso 140.0g e cor 0.55 é uma: Maçã
Peso 120.0g e cor 0.45 é uma: Maçã
Peso 135.0g e cor 0.52 é uma: Maçã
Peso 125.0g e cor 0.48 é uma: Maçã
```

